

View

FlagdemoApplication

+ main():

SQLiteDialect

+ SQLiteDialect()
- registerColumnType(int varchar, String s): void
+ getIdentityColumnSupport(): IdentityColumnSupport

HomeController

(no fields)

+ @GetMapping("/") home(): String

FlaggleView

FlaggleController

- CountryController {final}: countryController

+ @GetMapping("/") showLobby(): String
+ @GetMapping("/start") startGame(DifficultyLevel difficulty, Model model, HttpSession session): String
+ @PostMapping("/guess") guess(String countryName, String gameId, Model model, HttpSession session): String
+ @PostMapping("/giveup") giveUp(String gameId, Model model, HttpSession session): String
- buildGuessList(FlaggleViewModel viewModel): List<

GlobeView

GlobeController

- CountryController {final}: countryController

+ @GetMapping("/") showLobby(): String
+ @GetMapping("/start") startGame(Model model, HttpSession session): String
+ @RequestMapping("/guess/ajax") @ResponseBody guessAjax(String countryName, String gameId, HttpSession session): GuessResultGlobeBL
+ @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String gameId, HttpSession session): Map
+ @PostMapping("/hint/ajax") @ResponseBody hintAjax(String gameId, HttpSession session): Map
+ @GetMapping("/countries") @ResponseBody getCountries(String gameId, HttpSession session): List
- isValidForGlobe(CountryBL country): boolean

MatchView

FlaggleBlitzController

- Set {static final}: VALID_DURATIONS_SECONDS
- CountryController {final}: countryController

+ @GetMapping("/lobby") lobby(int durationSeconds, Model model): String
+ @GetMapping("/start") start(DifficultyLevel difficulty, AiSkillLevel aiLevel, int durationSeconds, Model model, HttpSession session): String
- normalizeDuration(int durationSeconds): int
+ @PostMapping("/guess/ajax") @ResponseBody guessAjax(String matchId, String countryName, HttpSession session): Map
+ @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String matchId, HttpSession session): Map
+ @GetMapping("/status/ajax") @ResponseBody statusAjax(String matchId, HttpSession session): Map
+ @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String matchId, HttpSession session): Map
+ @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String matchId, HttpSession session): Map
- buildStatePayload(FlaggleBlitzViewModel viewModel, GuessResultBL lastGuess, String givenUpName): Map
- encodeToBase64(BufferedImage image): String

FlaggleMatchController

- Set {static final}: VALID_BEST_OF
- CountryController {final}: countryController

+ @GetMapping("/") modeSelect(): String
+ @GetMapping("/format") format(): String
+ @GetMapping("/lobby") lobby(int bestOf, Model model): String
+ @GetMapping("/start") start(DifficultyLevel difficulty, AiSkillLevel aiLevel, int bestOf, Model model, HttpSession session): String
- normalizeBestOf(int bestOf): int
+ @PostMapping("/guess/ajax") @ResponseBody guessAjax(String matchId, String countryName, HttpSession session): Map
+ @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String matchId, HttpSession session): Map
+ @GetMapping("/status/ajax") @ResponseBody statusAjax(String matchId, HttpSession session): Map
+ @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String matchId, HttpSession session): Map
+ @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String matchId, HttpSession session): Map
+ @PostMapping("/nextRound/ajax") @ResponseBody nextRoundAjax(String matchId, HttpSession session): Map
- buildStatePayload(FlaggleMatchViewModel viewModel, GuessResultBL lastGuess): Map
- encodeToBase64(BufferedImage image): String

FlaggleMultiplayerController

- Set {static final}: VALID_BEST_OF
- Set {static final}: VALID_DURATIONS_SECONDS
- CountryController {final}: countryController
- GameRoomRegistryBL {final}: registry

+ @GetMapping("/lobby") lobby(): String
+ @PostMapping("/create") create(DifficultyLevel difficulty, int bestOf, Model model, HttpSession session): String
+ @PostMapping("/blitz/create") createBlitz(DifficultyLevel difficulty, int durationSeconds, Model model, HttpSession session): String
+ @PostMapping("/join") join(String roomCode, Model model, HttpSession session): String
+ @GetMapping("/room") room(String roomCode, Model model, HttpSession session): String
- renderMatchScreen(PvpMatchRoom room, int slot, Model model): String
- normalizeBestOf(int bestOf): int
- normalizeDuration(int durationSeconds): int
+ @PostMapping("/guess/ajax") @ResponseBody guessAjax(String roomCode, String playerToken, String countryName): Map
+ @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String roomCode, String playerToken): Map
+ @PostMapping("/nextRound/ajax") @ResponseBody nextRoundAjax(String roomCode, String playerToken): Map
+ @GetMapping("/status/ajax") @ResponseBody statusAjax(String roomCode, String playerToken): Map
+ @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String roomCode, String playerToken): Map
+ @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String roomCode, String playerToken): Map
- buildStatePayload(PvpMatchViewModel viewModel, GuessResultBL lastGuess): Map
+ @PostMapping("/blitz/guess/ajax") @ResponseBody blitzGuessAjax(String roomCode, String playerToken, String countryName): Map
+ @PostMapping("/blitz/giveup/ajax") @ResponseBody blitzGiveUpAjax(String roomCode, String playerToken): Map
+ @GetMapping("/blitz/status/ajax") @ResponseBody blitzStatusAjax(String roomCode, String playerToken): Map
+ @PostMapping("/blitz/pause/ajax") @ResponseBody blitzPauseAjax(String roomCode, String playerToken): Map
+ @PostMapping("/blitz/resume/ajax") @ResponseBody blitzResumeAjax(String roomCode, String playerToken): Map
- buildBlitzStatePayload(PvpBlitzViewModel viewModel, GuessResultBL lastGuess, String givenUpName): Map
- mapWinner(PvpRoundWinner winner, int viewerSlot): String
- encodeToBase64(BufferedImage image): String

GlobeMatchView

GlobeBlitzController

- Set {static final}: VALID_DURATIONS_SECONDS
- CountryController {final}: countryController

+ @GetMapping("/lobby") lobby(int durationSeconds, Model model): String
+ @GetMapping("/start") start(AiSkillLevel aiLevel, int durationSeconds, Model model, HttpSession session): String
- normalizeDuration(int durationSeconds): int
+ @PostMapping("/guess/ajax") @ResponseBody guessAjax(String matchId, String countryName, HttpSession session): Map
+ @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String matchId, HttpSession session): Map
+ @GetMapping("/status/ajax") @ResponseBody statusAjax(String matchId, HttpSession session): Map
+ @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String matchId, HttpSession session): Map
+ @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String matchId, HttpSession session): Map
- buildStatePayload(GlobeBlitzViewModel viewModel, GuessResultGlobeBL lastGuess, String givenUpName): Map

GlobeMatchController

- Set {static final}: VALID_BEST_OF
- CountryController {final}: countryController

+ @GetMapping("") modeSelect(): String
+ @GetMapping("/format") format(): String
+ @GetMapping("/lobby") lobby(int bestOf, Model model): String
+ @GetMapping("/start") start(AiSkillLevel aiLevel, int bestOf, Model model, HttpSession session): String
- normalizeBestOf(int bestOf): int
+ @PostMapping("/guess/ajax") @ResponseBody guessAjax(String matchId, String countryName, HttpSession session): Map
+ @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String matchId, HttpSession session): Map
+ @GetMapping("/status/ajax") @ResponseBody statusAjax(String matchId, HttpSession session): Map
+ @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String matchId, HttpSession session): Map
+ @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String matchId, HttpSession session): Map
+ @PostMapping("/nextRound/ajax") @ResponseBody nextRoundAjax(String matchId, HttpSession session): Map
+ @PostMapping("/hint/ajax") @ResponseBody hintAjax(String matchId, HttpSession session): Map
- buildStatePayload(GlobeMatchViewModel viewModel, GuessResultGlobeBL lastGuess): Map

GlobeMultiplayerController

- Set {static final}: VALID_BEST_OF
- Set {static final}: VALID_DURATIONS_SECONDS
- CountryController {final}: countryController
- GlobeRoomRegistryBL {final}: registry

+ @GetMapping("/lobby") lobby(): String
+ @PostMapping("/create") create(int bestOf, Model model, HttpSession session): String
+ @PostMapping("/blitz/create") createBlitz(int durationSeconds, Model model, HttpSession session): String
+ @PostMapping("/join") join(String roomCode, Model model, HttpSession session): String
+ @GetMapping("/room") room(String roomCode, Model model, HttpSession session): String
- renderMatchScreen(GlobePvpMatchRoom room, int slot, Model model): String
- normalizeBestOf(int bestOf): int
- normalizeDuration(int durationSeconds): int
+ @PostMapping("/guess/ajax") @ResponseBody guessAjax(String roomCode, String playerToken, String countryName): Map
+ @PostMapping("/giveup/ajax") @ResponseBody giveUpAjax(String roomCode, String playerToken): Map
+ @PostMapping("/nextRound/ajax") @ResponseBody nextRoundAjax(String roomCode, String playerToken): Map
+ @PostMapping("/hint/ajax") @ResponseBody hintAjax(String roomCode, String playerToken): Map
+ @GetMapping("/status/ajax") @ResponseBody statusAjax(String roomCode, String playerToken): Map
+ @PostMapping("/pause/ajax") @ResponseBody pauseAjax(String roomCode, String playerToken): Map
+ @PostMapping("/resume/ajax") @ResponseBody resumeAjax(String roomCode, String playerToken): Map
- buildStatePayload(PvpGlobeMatchViewModel viewModel, GuessResultGlobeBL lastGuess): Map
+ @PostMapping("/blitz/guess/ajax") @ResponseBody blitzGuessAjax(String roomCode, String playerToken, String countryName): Map
+ @PostMapping("/blitz/giveup/ajax") @ResponseBody blitzGiveUpAjax(String roomCode, String playerToken): Map
+ @GetMapping("/blitz/status/ajax") @ResponseBody blitzStatusAjax(String roomCode, String playerToken): Map
+ @PostMapping("/blitz/pause/ajax") @ResponseBody blitzPauseAjax(String roomCode, String playerToken): Map
+ @PostMapping("/blitz/resume/ajax") @ResponseBody blitzResumeAjax(String roomCode, String playerToken): Map
- buildBlitzStatePayload(PvpGlobeBlitzViewModel viewModel, GuessResultGlobeBL lastGuess, String givenUpName): Map
- mapWinner(PvpRoundWinner winner, int viewerSlot): String

ViewModel

FlaggleVM

FlaggleViewModel

- FlaggleModel: fm

+ StartNewGame(): void
+ StartNewGame(DifficultyLevel difficulty): void
+ getDifficulty(): DifficultyLevel
+ Guess(String guessesCountryName): BufferedImage
+ getAttempts(): int
+ getAllCountries(): List
+ getGuesses(): List
+ getTargetCountry(): CountryBL
+ isCorrect(): boolean

GlobeVM

GlobeViewModel

- GlobeModel: gm

+ StartNewGame(): void
+ Guess(String guessedCountryName): GuessResultGlobeBL
+ useHint(): String
+ GetAttempts(): int
+ getHintsUsed(): int
+ getAllCountries(): List
+ getGuesses(): List
+ getTargetCountry(): CountryBL
+ isCorrect(): boolean

MatchVM

FlaggleBlitzViewModel

- FlaggleBlitzModel {final}: fm

+ startMatch(): void
+ humanGuess(String countryName): GuessResultBL
+ humanGiveUpFlag(): String
+ refreshAiProgress(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): FlaggleBlitzEngineBL
+ getAllCountries(): List

FlaggleMatchViewModel

- FlaggleMatchModel {final}: fm

+ startMatch(): void
+ advanceToNextRound(): void
+ humanGuess(String countryName): GuessResultBL
+ humanGiveUpRound(): void
+ refreshAiTimeout(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): FlaggleMatchEngineBL
+ getAllCountries(): List

PvpBlitzViewModel

- PvpMatchRoom {final}: room
- int {final}: mySlot

+ getEngine(): PvpBlitzEngineBL
+ submitGuess(String countryName): GuessResultBL
+ giveUpFlag(): String

PvpMatchViewModel

- PvpMatchRoom {final}: room
- int {final}: mySlot

+ getEngine(): PvpMatchEngineBL
+ submitGuess(String countryName): GuessResultBL
+ giveUpRound(): void
+ advanceToNextRound(): void
+ markReady(): void

GlobeMatchVM

GlobeBlitzViewModel

- GlobeBlitzModel {final}: gm

+ startMatch(): void
+ humanGuess(String countryName): GuessResultGlobeBL
+ humanGiveUpFlag(): String
+ refreshAiProgress(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): GlobeBlitzEngineBL
+ getAllCountries(): List

GlobeMatchViewModel

- GlobeMatchModel {final}: gm

+ startMatch(): void
+ advanceToNextRound(): void
+ humanGuess(String countryName): GuessResultGlobeBL
+ humanGiveUpRound(): void
+ refreshAiTimeout(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ useHint(): String
+ getEngine(): GlobeMatchEngineBL
+ getAllCountries(): List

PvpGlobeBlitzViewModel

- GlobePvpMatchRoom {final}: room
- int {final}: mySlot

+ getEngine(): PvpGlobeBlitzEngineBL
+ submitGuess(String countryName): GuessResultGlobeBL
+ giveUpFlag(): String

PvpGlobeMatchViewModel

- GlobePvpMatchRoom {final}: room
- int {final}: mySlot

+ getEngine(): PvpGlobeMatchEngineBL
+ submitGuess(String countryName): GuessResultGlobeBL
+ giveUpRound(): void
+ advanceToNextRound(): void
+ markReady(): void
+ useHint(): String

Model

FlaggleModel

FlaggleModel

- GameService: gs
- CountryBL: targetCountry
- List: allCountries
- List: guesses
- int: attempts

- + StartNewGame(): void
- + StartNewGame(DifficultyLevel difficulty): void
- + getDifficulty(): DifficultyLevel
- + Guess(String guessesCountryName): GuessResultBL
- + resetGuesses(): void

GlobeModel

GlobeModel

- GlobeGameService: gs
- CountryBL: targetCountry
- List: allCountries
- List: guesses
- int: attempts
- int: hintsUsed

- + StartNewGame(): void
- + Guess(String guessedCountryName): GuessResultGlobeBL
- + useHint(): String
- + resetGuesses(): void

MatchModel

FlaggleBlitzModel

- FlaggleBlitzService {final}: service
- List {final}: allCountries

- + startMatch(): void
- + humanGuess(String countryName): GuessResultBL
- + humanGiveUpFlag(): String
- + refreshAiProgress(): void
- + pauseMatch(): void
- + resumeMatch(): void
- + getEngine(): FlaggleBlitzEngineBL

FlaggleMatchModel

- FlaggleMatchService {final}: service
- List {final}: allCountries

- + startMatch(): void
- + advanceToNextRound(): void
- + humanGuess(String countryName): GuessResultBL
- + humanGiveUpRound(): void
- + refreshAiTimeout(): void
- + pauseMatch(): void
- + resumeMatch(): void
- + getEngine(): FlaggleMatchEngineBL

GlobeMatchModel

GlobeBlitzModel

- GlobeBlitzService {final}: service
- List {final}: allCountries

+ startMatch(): void
+ humanGuess(String countryName): GuessResultGlobeBL
+ humanGiveUpFlag(): String
+ refreshAiProgress(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ getEngine(): GlobeBlitzEngineBL

GlobeMatchModel

- GlobeMatchService {final}: service
- List {final}: allCountries

+ startMatch(): void
+ advanceToNextRound(): void
+ humanGuess(String countryName): GuessResultGlobeBL
+ humanGiveUpRound(): void
+ refreshAiTimeout(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ useHint(): String
+ getEngine(): GlobeMatchEngineBL

Service Layer

FlaggleSL

GameService

- GameEngineBL: geb

+ StartNewGame(): void
+ StartNewGame(DifficultyLevel difficulty): void
+ getDifficulty(): DifficultyLevel
+ Guess(String countryName): GuessResultBL

GlobeSL

GlobeGameService

- GlobeEngineBL: geb

+ StartNewGame(): void
+ Guess(String countryName): GuessResultGlobeBL
+ useHint(): String
+ getHintsUsed(): int

MatchSL

FlaggleBlitzService

- FlaggleBlitzEngineBL {final}: engine

+ startMatch(): void
+ humanGuess(String countryName): GuessResultBL
+ humanGiveUpFlag(): String
+ refreshAiProgress(): void
+ pauseMatch(): void
+ resumeMatch(): void

FlaggleMatchService

- FlaggleMatchEngineBL {final}: engine

+ startMatch(): void
+ advanceToNextRound(): void
+ humanGuess(String countryName): GuessResultBL
+ humanGiveUpRound(): void
+ refreshAiTimeout(): void
+ pauseMatch(): void
+ resumeMatch(): void

GlobeMatchSL

GlobeBlitzService

- GlobeBlitzEngineBL {final}: engine

+ startMatch(): void
+ humanGuess(String countryName): GuessResultGlobeBL
+ humanGiveUpFlag(): String
+ refreshAiProgress(): void
+ pauseMatch(): void
+ resumeMatch(): void

GlobeMatchService

- GlobeMatchEngineBL {final}: engine

+ startMatch(): void
+ advanceToNextRound(): void
+ humanGuess(String countryName): GuessResultGlobeBL
+ humanGiveUpRound(): void
+ refreshAiTimeout(): void
+ pauseMatch(): void
+ resumeMatch(): void
+ useHint(): String

Business Layer

CountryBL

- String: name
- int: ID
- String: flagPath
- double: latitude
- double: longitude
- String: neighbors
- String: iso3

- + getFlagImage(): BufferedImage
- + equals(Object o): boolean

FlaggleBL

GuessResultBL

- int {static final}: TOLERANCE
- Color {static final}: FRAME_COLOR
- Color {static final}: BACKGROUND_COLOR
- Color {static final}: RED
- Color {static final}: BLUE
- Color {static final}: YELLOW
- Color {static final}: GREEN
- Color {static final}: ORANGE
- Color {static final}: PURPLE
- Color {static final}: BLACK
- Color {static final}: WHITE
- Color {static final}: GRAY
- boolean {final}: isCorrect
- CountryBL {final}: guessedCountry
- CountryBL {final}: targetCountry
- BufferedImage {final}: flagDifferences
- String {final}: guessedFlagBase64
- String {final}: flagDifferencesBase64

- + toString(): String
- {static} encodeToBase64(BufferedImage image): String
- + {static} calculateFlagDifferences(BufferedImage guessed, BufferedImage target): BufferedImage
- {static} calculateAccumulatedEasyReveal(BufferedImage previousReveal, BufferedImage guessed, BufferedImage target): BufferedImage
- + {static} calculateMatchPercentage(BufferedImage diffImage, boolean easyMode): int
- {static} normalizeColor(Color c): Color
- {static} colorDistance(Color c1, Color c2): double
- {static} areColorsSimilar(Color c1, Color c2): boolean
- {static} isFrameOrBackground(Color c): boolean

GameEngineBL

- CountryBL: targetCountry
- int: attempts
- boolean: gameOver
- CountryController: cc
- DifficultyLevel: difficulty
- BufferedImage: revealedFlag

- + StartNewGame(): void
- + StartNewGame(DifficultyLevel difficulty): void
- + getDifficulty(): DifficultyLevel
- + Guess(String countryName): GuessResultBL
- selectRandomCountry(): CountryBL

DifficultyLevel (enum)

<> DifficultyLevel
EASY
HARD

GlobeBL

GuessResultGlobeBL

- boolean {final}: isCorrect
- CountryBL {final}: guessedCountry
- CountryBL {final}: targetCountry
- float {final}: distance
- ProximityLevel {final}: proximityLevel
- double {static final}: EARTH_RADIUS_KM

- + getColorHex(): String
- calculateProximityLevel(): ProximityLevel
- isNeighbor(): boolean
- calculateDistance(double lat1, double lon1, double lat2, double lon2): float
- + toString(): String

GlobeEngineBL

- CountryBL: targetCountry
- int: attempts
- boolean: gameOver
- CountryController: cc
- int: hintsUsed
- Set {final}: revealedLetterPositions

- + StartNewGame(): void
- + Guess(String countryName): GuessResultGlobeBL
- + useHint(): String
- + SelectRandomCountry(): CountryBL
- isValidForGlobe(CountryBL country): boolean
- + getAllCountries(): List

CountryNameHintUtil

- Random {static final}: RANDOM

- + {static} countRevealableLetters(String name): int
- + {static} revealRandomLetter(String name, Set revealedPositions): void
- + {static} maskName(String name, Set revealedPositions): String

ProximityLevel

<> ProximityLevel
CORRECT
NEIGHBOR
VERY_CLOSE
CLOSE
MEDIUM
FAR
VERY_FAR
EXTREME
- String {final}: colorHex

MatchBL

AiOpponentPlan

- List {final}: attemptTimestamps
- List {final}: displayedPercents

- + getSolveTimeSeconds(): double
- + getAttemptsSoFar(double elapsedSeconds): int
- + isSolved(double elapsedSeconds): boolean
- + getProgressPercent(double elapsedSeconds): int

ScoredCandidate

- CountryBL {final}: country
- int {final}: percent

BlitzFlagResult

- int {final}: order
- String {final}: countryName
- boolean {final}: solved
- int {final}: attempts
- double {final}: timeTakenSeconds

AiOpponentPlanFactory

- Random {static final}: RANDOM

- + {static} createPlan(AiSkillLevel level, CountryController cc, CountryBL target, DifficultyLevel flagRevealDifficulty): AiOpponentPlan
- {static} pickCandidates(List pool, CountryBL target, int count, int sampleSize): List
- {static} computeDisplayedPercents(List candidates, CountryBL target, DifficultyLevel flagRevealDifficulty): List
- {static} hardMatchPercent(CountryBL guessed, CountryBL target): int

AiSkillLevel

<> AiSkillLevel
EASY
MEDIUM
HARD

FlaggleMatchEngineBL
- CountryController {final}: cc - DifficultyLevel {final}: difficulty - AiSkillLevel {final}: aiLevel - int {final}: pointsToWin - int: humanScore - int: aiScore - int: roundNumber - boolean: matchOver - long: matchStartTimeMillis - long: roundStartTimeMillis - boolean: paused - long: pauseStartedAtMillis - long: pausedMillisThisRound - long: pausedMillisTotalMatch - CountryBL: currentTarget - int: humanAttemptsThisRound - BufferedImage: humanRevealedFlag - RoundWinner: currentRoundWinner - AiOpponentPlan: aiPlan - List {final}: roundHistory
+ {synchronized} startMatch(): void + {synchronized} advanceToNextRound(): void - startNextRound(): void - selectRandomCountry(): CountryBL + {synchronized} humanGuess(String countryName): GuessResultBL + {synchronized} humanGiveUpRound(): void + {synchronized} refreshAiTimeout(): void - finishRound(): void + {synchronized} pauseMatch(): void + {synchronized} resumeMatch(): void + getRoundElapsedSeconds(): double + getMatchElapsedSeconds(): double + getBestOf(): int + isRoundOver(): boolean + getMatchWinner(): RoundWinner

FlaggleBlitzEngineBL
- CountryController {final}: cc - DifficultyLevel {final}: difficulty - AiSkillLevel {final}: aiLevel - double {final}: durationSeconds - List: queue - long: matchStartTimeMillis - boolean: matchOver - boolean: paused - long: pauseStartedAtMillis - long: pausedMillisTotal - int: humanQueueIndex - int: humanAttemptsThisFlag - double: humanCurrentFlagStartElapsed - BufferedImage: humanRevealedFlag - int: humanCorrectCount - List {final}: humanHistory - int: aiQueueIndex - double: aiCurrentFlagStartElapsed - AiOpponentPlan: aiPlan - int: aiCorrectCount - List {final}: aiHistory
+ {synchronized} startMatch(): void + {synchronized} humanGuess(String countryName): GuessResultBL + {synchronized} humanGiveUpFlag(): String - advanceHumanToNextFlag(): void + {synchronized} refreshAiProgress(): void + {synchronized} pauseMatch(): void + {synchronized} resumeMatch(): void + getElapsedSeconds(): double + getTimeRemainingSeconds(): double + getCurrentHumanTarget(): CountryBL + getQueueFlagAt(int index): CountryBL + getAiElapsedOnCurrentFlag(): double + getMatchWinner(): RoundWinner

GameRoomRegistryBL
- long {static final}: STALE_ROOM_MILLIS - String {static final}: CODE_CHARS - int {static final}: CODE_LENGTH - CountryController {final}: cc - Map {final}: rooms - Random {final}: random
+ createRoom(DifficultyLevel difficulty, int bestOf, String player1Token): PvpMatchRoom + createBlitzRoom(DifficultyLevel difficulty, int durationSeconds, String player1Token): PvpMatchRoom - generateCode(): String + getRoom(String roomCode): PvpMatchRoom + removeRoom(String roomCode): void + sweepStaleRooms(): void

PvpGameMode
<> PvpGameMode - BEST_OF_N - BLITZ

MatchRoundResult
- int {final}: roundNumber - RoundWinner {final}: winner - String {final}: targetCountryName - int {final}: humanAttempts - int {final}: aiAttempts

PvpRoundResult
- int {final}: roundNumber - PvpRoundWinner {final}: winner - String {final}: targetCountryName - int {final}: player1Attempts - int {final}: player2Attempts

RoundWinner (enum)
<> RoundWinner NONE HUMAN AI DRAW

PvpMatchEngineBL
- CountryController {final}: cc - DifficultyLevel {final}: difficulty - int {final}: pointsToWin - int: score1 - int: score2 - int: roundNumber - boolean: matchOver - long: matchStartTimeMillis - long: roundStartTimeMillis - long {static final}: PAUSE_MAX_MILLIS - boolean: paused - int: pausedBySlot - long: pauseStartedAtMillis - long: pausedMillisThisRound - long: pausedMillisTotalMatch - boolean: pause1Used - boolean: pause2Used - long {static final}: READY_MAX_MILLIS - boolean: ready1 - boolean: ready2 - long: roundOverAtMillis - CountryBL: currentTarget - int: attempts1 - int: attempts2 - BufferedImage: revealedFlag1 - BufferedImage: revealedFlag2 - int: progressPercent1 - int: progressPercent2 - PvpRoundWinner: currentRoundWinner - List {final}: roundHistory
+ {synchronized} startMatch(): void + {synchronized} advanceToNextRound(): void - startNextRound(): void - selectRandomCountry(): CountryBL + {synchronized} submitGuess(int slot, String countryName): GuessResultBL + {synchronized} giveUpRound(int slot): void - finishRound(): void + {synchronized} markReady(int slot): boolean + isReady(int slot): boolean - readyDeadlinePassed(): boolean + getReadyRemainingMillis(): long + {synchronized} pauseMatch(int slot): boolean + {synchronized} resumeMatch(): void + {synchronized} refreshPauseTimeout(): void + hasUsedPause(int slot): boolean + getPauseRemainingMillis(): long + getRoundElapsedSeconds(): double + getMatchElapsedSeconds(): double + getScore(int slot): int + getAttempts(int slot): int + getProgressPercent(int slot): int + getBestOf(): int + isRoundOver(): boolean + getMatchWinner(): PvpRoundWinner

PvpMatchRoom
- String {final}: roomCode - DifficultyLevel {final}: difficulty - PvpGameMode {final}: gameMode - int {final}: bestOf - int {final}: pointsToWin - int {final}: durationSeconds - CountryController {final}: cc - String {final}: player1Token - String {volatile}: player2Token - PvpRoomStatus {volatile}: status - long {volatile}: lastActivityMillis - PvpMatchEngineBL {volatile}: engine - PvpBlitzEngineBL {volatile}: blitzEngine
+ {static} createBestOf(String roomCode, DifficultyLevel difficulty, int bestOf, String player1Token, CountryController cc): PvpMatchRoom + {static} createBlitz(String roomCode, DifficultyLevel difficulty, int durationSeconds, String player1Token, CountryController cc): PvpMatchRoom + {synchronized} joinPlayer2(String token): boolean + slotForToken(String token): int + touch(): void + isStale(long staleMillis): boolean + getStatus(): PvpRoomStatus

PvpBlitzEngineBL
- CountryController {final}: cc - DifficultyLevel {final}: difficulty - double {final}: durationSeconds - List: queue - long: matchStartTimeMillis - boolean: matchOver - long {static final}: PAUSE_MAX_MILLIS - boolean: paused - int: pausedBySlot - long: pauseStartedAtMillis - long: pausedMillisTotal - boolean: pause1Used - boolean: pause2Used - int: queueIndex1 - int: attemptsThisFlag1 - double: currentFlagStartElapsed1 - BufferedImage: revealedFlag1 - int: progressPercent1 - int: correctCount1 - List {final}: history1 - int: queueIndex2 - int: attemptsThisFlag2 - double: currentFlagStartElapsed2 - BufferedImage: revealedFlag2 - int: progressPercent2 - int: correctCount2 - List {final}: history2
+ {synchronized} startMatch(): void + {synchronized} submitGuess(int slot, String countryName): GuessResultBL + {synchronized} giveUpFlag(int slot): String - advanceToNextFlag(int slot): void + {synchronized} refreshState(): void + {synchronized} pauseMatch(int slot): boolean + {synchronized} resumeMatch(): void + {synchronized} refreshPauseTimeout(): void + hasUsedPause(int slot): boolean + getPauseRemainingMillis(): long + getElapsedSeconds(): double + getTimeRemainingSeconds(): double + getCorrectCount(int slot): int + getAttemptsThisFlag(int slot): int + getProgressPercent(int slot): int + getHistory(int slot): List + getCurrentTarget(int slot): CountryBL + getQueueFlagAt(int index): CountryBL + getMatchWinner(): PvpRoundWinner

PvpRoomStatus (enum)

```
<> PvpRoomStatus  
WAITING_FOR_OPPONENT  
IN_PROGRESS  
FINISHED
```

PvpRoundWinner (enum)

```
<> PvpRoundWinner  
NONE  
PLAYER1  
PLAYER2  
DRAW
```

GlobeMatchBL

GlobeAiOpponentPlan

```
- List {final}: attemptTimestamps  
- List {final}: displayedProximity  
  
+ getSolveTimeSeconds(): double  
+ getAttemptsSoFar(double elapsedSeconds): int  
+ isSolved(double elapsedSeconds): boolean  
+ getProgressLevel(double elapsedSeconds): ProximityLevel
```

GlobeMatchEngineBL

```
- CountryController {final}: cc  
- AiSkillLevel {final}: aiLevel  
- int {final}: pointsToWin  
- int: humanScore  
- int: aiScore  
- int: roundNumber  
- boolean: matchOver  
- long: matchStartTimeMillis  
- long: roundStartTimeMillis  
- boolean: paused  
- long: pauseStartedAtMillis  
- long: pausedMillisThisRound  
- long: pausedMillisTotalMatch  
- int {static final}: GRACE_SECONDS_PER_HINT  
- int: hintsUsedThisRound  
- Set {final}: revealedLetterPositions  
- CountryBL: currentTarget  
- int: humanAttemptsThisRound  
- RoundWinner: currentRoundWinner  
- GlobeAiOpponentPlan: aiPlan  
- RoundWinner: provisionalWinner  
- double: graceDeadlineElapsedSeconds  
- double: lastRoundDurationSeconds  
- List {final}: roundHistory  
  
+ {synchronized} startMatch(): void  
+ {synchronized} advanceToNextRound(): void  
- startNextRound(): void  
- selectRandomCountry(): CountryBL  
+ {synchronized} humanGuess(String countryName):  
GuessResultGlobeBL  
+ {synchronized} humanGiveUpRound(): void  
+ {synchronized} useHint(): String  
+ getHintMaskedName(): String  
+ getGraceRemainingSeconds(): double  
+ {synchronized} refreshAiTimeout(): void  
- finishRound(): void  
+ {synchronized} pauseMatch(): void  
+ {synchronized} resumeMatch(): void  
+ getRoundElapsedSeconds(): double  
+ getMatchElapsedSeconds(): double  
+ getBestOf(): int  
+ isRoundOver(): boolean  
+ getMatchWinner(): RoundWinner  
+ getAiProgressLevel(): ProximityLevel
```

GlobeAiOpponentPlanFactory

```
- Random {static final}: RANDOM  
  
+ {static} createPlan(AiSkillLevel level, CountryController cc,  
CountryBL target): GlobeAiOpponentPlan  
- {static} sampleAttempts(int minAttempts, int maxAttempts): int  
- {static} validPool(CountryController cc, CountryBL target): List  
- {static} pickCandidates(List pool, CountryBL target, int count, int  
sampleSize): List  
- {static} isCloser(GuessResultGlobeBL a, GuessResultGlobeBL  
b): boolean  
- {static} computeDisplayedProximity(List candidates, CountryBL  
target): List
```

GlobeBlitzEngineBL

```
- CountryController {final}: cc  
- AiSkillLevel {final}: aiLevel  
- double {final}: durationSeconds  
- List: queue  
- long: matchStartTimeMillis  
- boolean: matchOver  
- boolean: paused  
- long: pauseStartedAtMillis  
- long: pausedMillisTotal  
- int: humanQueueIndex  
- int: humanAttemptsThisFlag  
- double: humanCurrentFlagStartElapsed  
- int: humanCorrectCount  
- List {final}: humanHistory  
- int: aiQueueIndex  
- double: aiCurrentFlagStartElapsed  
- GlobeAiOpponentPlan: aiPlan  
- int: aiCorrectCount  
- List {final}: aiHistory  
  
+ {synchronized} startMatch(): void  
+ {synchronized} humanGuess(String countryName):  
GuessResultGlobeBL  
+ {synchronized} humanGiveUpFlag(): String  
- advanceHumanToNextFlag(): void  
+ {synchronized} refreshAiProgress(): void  
+ {synchronized} pauseMatch(): void  
+ {synchronized} resumeMatch(): void  
+ getElapsedSeconds(): double  
+ getTimeRemainingSeconds(): double  
+ getCurrentHumanTarget(): CountryBL  
+ getQueueFlagAt(int index): CountryBL  
+ getAiElapsedOnCurrentFlag(): double  
+ getAiProgressLevel(): ProximityLevel  
+ getMatchWinner(): RoundWinner
```

ScoredCandidate
- CountryBL {final}: country - ProximityLevel {final}: level - float {final}: distance

GlobeRoomRegistryBL
- long {static final}: STALE_ROOM_MILLIS - String {static final}: CODE_CHARS - int {static final}: CODE_LENGTH - CountryController {final}: cc - Map {final}: rooms - Random {final}: random
+ createRoom(int bestOf, String player1Token): GlobePvpMatchRoom + createBlitzRoom(int durationSeconds, String player1Token): GlobePvpMatchRoom - generateCode(): String + getRoom(String roomCode): GlobePvpMatchRoom + removeRoom(String roomCode): void + sweepStaleRooms(): void

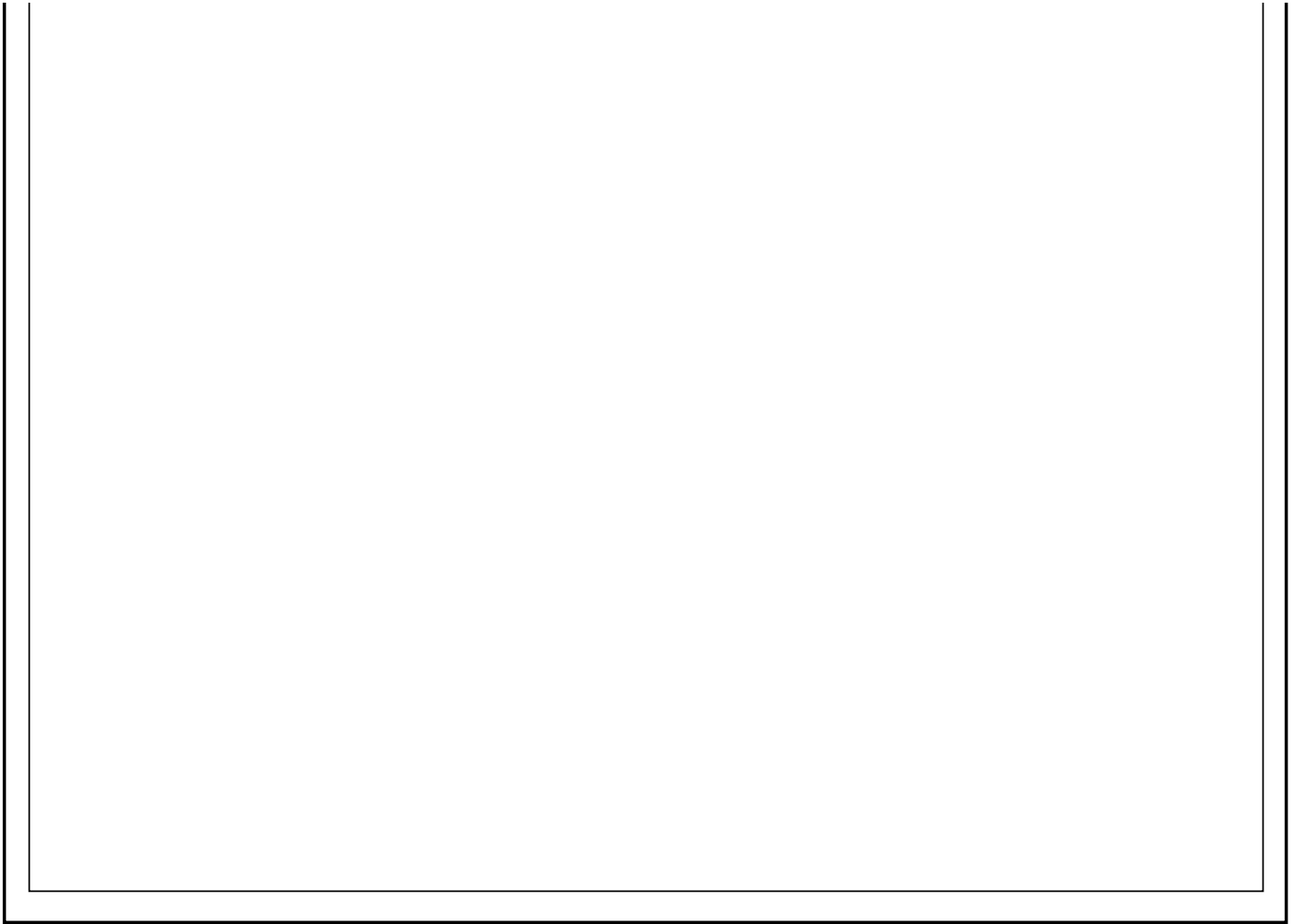
GlobePvpMatchRoom
- String {final}: roomCode - PvpGameMode {final}: gameMode - int {final}: bestOf - int {final}: pointsToWin - int {final}: durationSeconds - CountryController {final}: cc - String {final}: player1Token - String {volatile}: player2Token - PvpRoomStatus {volatile}: status - long {volatile}: lastActivityMillis - PvpGlobeMatchEngineBL {volatile}: engine - PvpGlobeBlitzEngineBL {volatile}: blitzEngine
+ {static} createBestOf(String roomCode, int bestOf, String player1Token, CountryController cc): GlobePvpMatchRoom + {static} createBlitz(String roomCode, int durationSeconds, String player1Token, CountryController cc): GlobePvpMatchRoom + {synchronized} joinPlayer2(String token): boolean + slotForToken(String token): int + touch(): void + isStale(long staleMillis): boolean + getStatus(): PvpRoomStatus

PvpGlobeBlitzEngineBL
- CountryController {final}: cc - double {final}: durationSeconds - List: queue - long: matchStartTimeMillis - boolean: matchOver - long {static final}: PAUSE_MAX_MILLIS - boolean: paused - int: pausedBySlot - long: pauseStartedAtMillis - long: pausedMillisTotal - boolean: pause1Used - boolean: pause2Used - int {static final}: NONE_ORDINAL - int: queueIndex1 - int: attemptsThisFlag1 - double: currentFlagStartElapsed1 - int: bestOrdinal1 - int: correctCount1 - List {final}: history1 - int: queueIndex2 - int: attemptsThisFlag2 - double: currentFlagStartElapsed2 - int: bestOrdinal2 - int: correctCount2 - List {final}: history2
+ {synchronized} startMatch(): void + {synchronized} submitGuess(int slot, String countryName): GuessResultGlobeBL + {synchronized} giveUpFlag(int slot): String - advanceToNextFlag(int slot): void + {synchronized} refreshState(): void + {synchronized} pauseMatch(int slot): boolean + {synchronized} resumeMatch(): void + {synchronized} refreshPauseTimeout(): void + hasUsedPause(int slot): boolean + getPauseRemainingMillis(): long + getElapsedSeconds(): double + getTimeRemainingSeconds(): double + getCorrectCount(int slot): int + getAttemptsThisFlag(int slot): int + getProgressLevel(int slot): ProximityLevel + getHistory(int slot): List + getCurrentTarget(int slot): CountryBL + getQueueFlagAt(int index): CountryBL + getMatchWinner(): PvpRoundWinner

PvpGlobeMatchEngineBL

- CountryController {final}: cc
- int {final}: pointsToWin
- int: score1
- int: score2
- int: roundNumber
- boolean: matchOver
- long: matchStartTimeMillis
- long: roundStartTimeMillis
- long {static final}: PAUSE_MAX_MILLIS
- boolean: paused
- int: pausedBySlot
- long: pauseStartedAtMillis
- long: pausedMillisThisRound
- long: pausedMillisTotalMatch
- boolean: pause1Used
- boolean: pause2Used
- long {static final}: READY_MAX_MILLIS
- boolean: ready1
- boolean: ready2
- long: roundOverAtMillis
- int {static final}: GRACE_SECONDS_PER_HINT
- int: hints1UsedThisRound
- int: hints2UsedThisRound
- Set {final}: revealedLetterPositions1
- Set {final}: revealedLetterPositions2
- CountryBL: currentTarget
- int: attempts1
- int: attempts2
- double: lastRoundDurationSeconds
- PvpRoundWinner: provisionalWinner
- double: graceDeadlineElapsedSeconds
- int {static final}: NONE_ORDINAL
- int: bestOrdinal1
- int: bestOrdinal2
- PvpRoundWinner: currentRoundWinner
- List {final}: roundHistory

- + {synchronized} startMatch(): void
- + {synchronized} advanceToNextRound(): void
- startNextRound(): void
- selectRandomCountry(): CountryBL
- + {synchronized} submitGuess(int slot, String countryName): GuessResultGlobeBL
- + {synchronized} giveUpRound(int slot): void
- finishRound(): void
- + {synchronized} markReady(int slot): boolean
- + isReady(int slot): boolean
- readyDeadlinePassed(): boolean
- + getReadyRemainingMillis(): long
- + {synchronized} useHint(int slot): String
- + getHintMaskedName(int slot): String
- + getHintsUsedThisRound(int slot): int
- + getGraceRemainingSeconds(): double
- + {synchronized} refreshGraceTimeout(): void
- + {synchronized} pauseMatch(int slot): boolean
- + {synchronized} resumeMatch(): void
- + {synchronized} refreshPauseTimeout(): void
- + hasUsedPause(int slot): boolean
- + getPauseRemainingMillis(): long
- + getRoundElapsedSeconds(): double
- + getMatchElapsedSeconds(): double
- + getScore(int slot): int
- + getAttempts(int slot): int
- + getProgressLevel(int slot): ProximityLevel
- + getBestOf(): int
- + isRoundOver(): boolean
- + getMatchWinner(): PvpRoundWinner



Data Access Layer

CountryController

- CountryRepository: cr

- getConnection(): Connection
- fixImagePath(String dbPath): String
+ getCountryById(int id): CountryBL
+ getCountryByName(String countryName): CountryBL
+ getAllCountries(): List
+ getNumberOfAllCountries(): int
- CheckCountryNameExists(String countryName): boolean

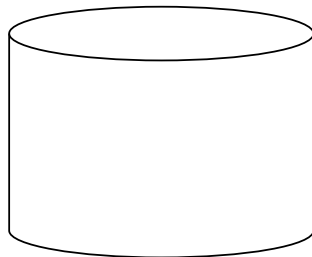
CountryDAL

- int: ID
- String: countryName
- String: code
- String: flagPath
- double: latitude
- double: longitude
- String: neighborsList
- String: iso3

CountryRepository

- Map: codeToName
- Map: nameToCode
- List: allCountries

- getConnection(): Connection
- loadCountries(): void
+ getNameByCode(String code): String
+ getCodeByName(String name): String
+ getNumberOfCountries(): int



Countries	
PK	<u>ID</u>
	<u>CountryName</u>
	<u>Code</u>
	<u>FlagPath</u>
	<u>Latitude</u>
	<u>Longitude</u>
	<u>NeighborCodes</u>